Rapport de Projet
Programmation Internet
Système de Partage de Fichiers P2P

Guillaume Xue : 22101031
David Yu : 22110478

19 janvier 2026

Table des matières

1 Introduction **2**
1.1 Objectifs du projet 2
1.2 Architecture générale 2

2 Implementation minimum **3**
2.1 Enregistrement serveur central 3
2.1.1 Enregistrement de la clé publique 3
2.1.2 Enregistrement de l'adresse IP et du port 3
2.1.3 Maintien de la connexion 4
2.2 Partage de fichiers et répertoires 6
2.2.1 Structure de l'arbre de Merkle 6
2.2.2 Réponse aux requêtes 7
2.3 Téléchargement de fichiers 7
2.3.1 Architecture du downloader 7
2.3.2 Fenêtre glissante 7
2.3.3 Contrôle de congestion adaptatif 8
2.3.4 Gestion des timeouts et retransmissions 9
2.3.5 Téléchargement récursif 9

3 Fonctionnalités avancées **10**
3.1 Traversal NAT 10
3.1.1 Principe du NAT traversal 10
3.1.2 Protocole de traversal 10
3.1.3 Détection du succès 11
3.1.4 Gestion des relais 12
3.2 Interface utilisateur 12
3.3 Gestion multi-protocoles IPv4/IPv6 12

Chapitre 1

Introduction

Ce projet implémente un système de partage de fichiers pair-à-pair (P2P) utilisant un serveur central pour la découverte des pairs et l'échange de clés publiques. Le système utilise le protocole UDP pour toutes les communications et implémente un arbre de Merkle pour garantir l'intégrité des données échangées.

1.1 Objectifs du projet

L'objectif principal est de créer un client capable de :
— S'enregistrer auprès d'un serveur central et maintenir cette connexion indéfiniment
— Partager des fichiers et répertoires avec d'autres pairs
— Télécharger des fichiers depuis d'autres pairs
— Fonctionner derrière un NAT grâce à des mécanismes de traversal
— Gérer efficacement les transferts avec fenêtre glissante et contrôle de congestion

1.2 Architecture générale

L'implémentation est réalisée en Go et s'articule autour de plusieurs composants clés :
— **Module transport** : Gestion des communications UDP, connexion au serveur, échange de paquets
— **Module merkle** : Construction et validation des arbres de Merkle pour les fichiers et répertoires
— **Module crypto** : Génération et gestion des clés ECDSA, signatures numériques
— **Module peer** : Gestion de la liste des pairs connectés et de leurs adresses
— **Module protocol** : Définition des types de messages et encodage/décodage des paquets

Chapitre 2

Implementation minimum

2.1 Enregistrement serveur central

2.1.1 Enregistrement de la clé publique

Pour le processus d'enregistrement au serveur central, nous avons d'abord l'enregistrement de la clé publique. Nous avons encodé notre clé privée en clé publique, puis nous avons envoyé une requête PUT au serveur central avec la clé publique dans le corps de la requête. Le serveur central stocke cette clé pour permettre aux autres clients de la récupérer ultérieurement.

2.1.2 Enregistrement de l'adresse IP et du port

L'enregistrement de l'adresse IP et du port UDP auprès du serveur central en Dual-Stack est géré par la fonction `TryConnectWithFallback`. Cette fonction tente d'abord d'ouvrir un socket UDP en mode Dual-Stack (IPv6 avec support IPv4). Si cette tentative échoue, elle bascule automatiquement en mode IPv4 pur.

Dual-Stack avec Fallback

Notre fonction `TryConnectWithFallback` implémente une méthode de connexion robuste en deux étapes :

Étape 1 : Ouverture du socket local Le client tente d'abord d'ouvrir un socket en mode Dual-Stack (IPv6 avec support IPv4) :

```
addrV6, _ := net.ResolveUDPAddr("udp", "[::]:0")
conn, err = net.ListenUDP("udp", addrV6)
```

Si cette tentative échoue (notamment sur certaines configurations Windows ou WSL), le système bascule automatiquement en mode IPv4 pur :

```
addrV4, _ := net.ResolveUDPAddr("udp4", "0.0.0.0:0")
conn, err = net.ListenUDP("udp4", addrV4)
```

Le port local est laissé à 0 pour que le système d'exploitation choisisse un port disponible, évitant ainsi les conflits "Address already in use".

Étape 2 : Handshake avec le serveur Une fois le socket local ouvert, le client détermine quelle adresse du serveur utiliser. En mode Dual-Stack, le client privilégie d'abord l'IPv6, puis tente l'IPv4 en fallback :

```
if isDualStack {
    targets = append(targets, protocol.GetServerUDPv6())
```

```
}
targets = append(targets, protocol.GetServerUDPv4())
```

Pour chaque adresse cible, le client :

1. Envoie un message `Hello` contenant son nom et signé avec sa clé privée
2. Attend une réponse du serveur pendant 2 secondes maximum
3. Vérifie que la réponse correspond bien à sa requête via l'ID du message

Gestion de la réponse serveur

La fonction `waitResponse` implémente un mécanisme de timeout robuste :

```
deadline := time.Now().Add(2 * time.Second)
conn.SetReadDeadline(deadline)
```

Elle lit les paquets entrants en boucle et vérifie que :
— L'ID du paquet correspond à celui envoyé (pour éviter les confusions)
— Le type de message est valide (`HelloReply`, `Ok`, ou `Ping`)

Optimisation Dual-Stack

Lorsque la connexion est établie en mode Dual-Stack via IPv6, le client envoie également un ping sur l'adresse IPv4 du serveur :

```
if isDualStack && targetStr == protocol.GetServerUDPv6() {
    if otherAddr, err := net.ResolveUDPAddr("udp",
                          protocol.GetServerUDPv4()); err == nil {
        SendHello(conn, otherAddr, name, privKey)
    }
}
```

Cette technique permet au serveur d'enregistrer les deux adresses du client (IPv4 et IPv6), garantissant ainsi une connectivité maximale avec les autres peers.

2.1.3 Maintien de la connexion

Une fois la connexion établie avec le serveur central et les autres pairs, il est essentiel de maintenir ces connexions actives pour éviter les timeouts et les déconnexions silencieuses. Notre implémentation utilise un mécanisme de keep-alive basé sur des pings périodiques.

Mécanisme de Keep-Alive

La fonction `KeepAlive` s'exécute en arrière-plan dans une goroutine dédiée et envoie régulièrement des pings à tous les pairs connectés. Elle utilise un ticker pour déclencher ces pings à intervalle régulier (typiquement 30 secondes) :

```
func (s *Server) KeepAlive(serverAddr *net.UDPAddr,
                           interval time.Duration, ctx context.Context) {
    ticker := time.NewTicker(interval)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return
        case <-s.shutdown:
            return
```

```
        case <-ticker.C:
            // Logique de ping...
        }
    }
}
```

Stratégie de Ping Multi-Cibles

À chaque cycle de keep-alive, le client effectue trois types de pings distincts :

1. Ping du serveur central Le client ping d'abord le serveur central sur ses deux adresses (IPv6 et IPv4 si disponibles). Les adresses sont ré-résolues à chaque fois pour gérer les potentiels changements d'IP DNS :

```
// Ping IPv6
if addr, err := net.ResolveUDPAddr("udp",
                protocol.GetServerUDPv6()); err == nil {
    SendPing(s.Conn, addr)
    time.Sleep(1 * time.Microsecond) // Garantir un ID unique
}

// Ping IPv4
if addr, err := net.ResolveUDPAddr("udp",
                protocol.GetServerUDPv4()); err == nil {
    SendPing(s.Conn, addr)
    time.Sleep(1 * time.Microsecond)
}
```

Le délai d'une microseconde entre chaque ping garantit que chaque message aura un ID unique basé sur le timestamp.

2. Ping des pairs connectés Ensuite, le client ping tous les pairs du réseau P2P avec lesquels il a établi une connexion. Pour chaque pair, toutes ses adresses connues (IPv4 et IPv6) sont pingées :

```
connectedPeers := s.PeerManager.List()
for _, peerName := range connectedPeers {
    if peerInfo, ok := s.PeerManager.Get(peerName); ok
       && peerInfo.Name != "jch.irif.fr" {
        for _, addr := range peerInfo.Addrs {
            SendPing(s.Conn, addr)
            time.Sleep(1 * time.Microsecond)
        }
    }
}
```

Cette approche garantit que :

- Les pairs restent conscients de notre présence active
- Les NAT/firewalls maintiennent les mappings de port ouverts
- Les changements d'adresse sont détectés rapidement

3. Nettoyage des pairs expirés Après l'envoi des pings, le client nettoie sa liste de pairs pour supprimer ceux qui n'ont pas répondu depuis un certain temps :

```
s.PeerManager.CleanExpired()
```

Cette opération évite l'accumulation de pairs morts dans la liste et libère les ressources associées.

Gestion des réponses Ping

Lorsqu'un pair reçoit un message `Ping`, il répond immédiatement avec un message `Ok` portant le même ID :

```
case protocol.Ping:
    SendOk(s.Conn, addr, pkt.Header.ID)
```

La réception de tout message (pas seulement `Ok`) d'un pair met à jour son timestamp d'activité :

```
if p, ok := s.PeerManager.GetByAddr(addr); ok {
    s.PeerManager.AddOrUpdate(p.Name, addr,
                              p.PublicKey, p.IsRelay)
}
```

Cette approche permet de considérer toute activité réseau comme preuve de vie, pas seulement les réponses aux pings explicites.

2.2 Partage de fichiers et répertoires

2.2.1 Structure de l'arbre de Merkle

Notre implémentation utilise un arbre de Merkle pour représenter les fichiers et répertoires partagés. Chaque nœud de l'arbre est identifié par le hash SHA-256 de son contenu, garantissant l'intégrité des données.

Types de nœuds

Le protocole définit quatre types de nœuds :

- **Type 0 (Chunk)** : Données brutes d'un fichier (max 1024 octets)
- **Type 1 (Directory)** : Répertoire simple (max 16 entrées)
- **Type 2 (Big)** : Fichier fragmenté en plusieurs chunks (max 32 enfants)
- **Type 3 (BigDirectory)** : Répertoire fragmenté (max 32 sous-répertoires)

Construction d'un chunk

Un chunk de type 0 est construit en préfixant les données avec le type :

```
func CreateChunk(data []byte) ([]byte, [32]byte) {
    if len(data) > MaxChunkSize {
        panic("Taille du chunk trop grande")
    }
    datum := make([]byte, 1+len(data))
    datum[0] = TypeChunk
    copy(datum[1:], data)
    return datum, sha256.Sum256(datum)
}
```

Construction d'un répertoire simple

Un répertoire de type 1 contient jusqu'à 16 entrées, chacune composée d'un nom (32 octets) et d'un hash (32 octets) :

```
func CreateDirectoryNode(entries []DirEntry) ([]byte, [32]byte) {
    if len(entries) > MaxDirEntries {
        panic("Trop d'entrees dans le repertoire")
```

```go
    }
    datum := make([]byte, 1+len(entries)*DirEntrySize)
    datum[0] = TypeDirectory
    for i, e := range entries {
        offset := 1 + i*DirEntrySize
        copy(datum[offset:], e.Name[:])
        copy(datum[offset+FileNameSize:], e.Hash[:])
    }
    return datum, sha256.Sum256(datum)
}
```

Cette structure permet de partager des répertoires de moins de 16 entrées conformément aux exigences minimales.

2.2.2 Réponse aux requêtes

Lorsqu'un pair demande un datum (fichier ou répertoire), le serveur vérifie d'abord dans ses stores locaux :

```go
func (s *Server) onDatumRequest(pkt *protocol.Packet, addr *net.UDPAddr) {
    hash, err := protocol.DecodeHashBody(pkt.Body)
    if err != nil {
        return
    }

    // Chercher d'abord en local, puis dans le cache telecharge
    data, ok := s.MerkleStore.Get(hash)
    if !ok {
        data, ok = s.Downloads.Get(hash)
    }

    if ok {
        SendDatum(s.Conn, addr, hash, data, pkt.Header.ID)
    } else {
        SendNoDatum(s.Conn, addr, hash, s.PrivKey, pkt.Header.ID)
    }
}
```

Le système maintient deux stores distincts :

— `MerkleStore` : Fichiers partagés localement

— `Downloads` : Fichiers téléchargés depuis d'autres pairs

2.3 Téléchargement de fichiers

2.3.1 Architecture du downloader

Le téléchargement est géré par le module `Downloader`, qui implémente une fenêtre glissante avec contrôle de congestion adaptatif. L'architecture repose sur trois workers concurrents :

1. **SenderLoop** : Envoie les requêtes de datums en respectant la taille de fenêtre
2. **ResponseLoop** : Traite les réponses reçues et ajuste la fenêtre
3. **MonitorLoop** : Gère les timeouts et retransmissions

2.3.2 Fenêtre glissante

La fenêtre glissante limite le nombre de requêtes en vol simultanées, évitant ainsi la congestion du réseau :

```go
// Worker 1 : Envoie les demandes
func (d *Downloader) senderLoop() {
    for d.running {
        // 1. Check Window
        d.mu.Lock()
        canSend := len(d.pending) < d.windowSize
        d.mu.Unlock()

        if !canSend {
            time.Sleep(10 * time.Millisecond)
            continue
        }

        // 2. Get Job
        hash, ok := <-d.workCh
        if !ok {
            return
        }

        // 3. Check si on l'a deja
        if _, exists := d.server.Downloads.Get(hash); exists {
            continue
        }

        // 4. Check si deja en cours
        d.mu.Lock()
        if _, inflight := d.pending[hash]; inflight {
            d.mu.Unlock()
            continue
        }
        d.pending[hash] = time.Now()
        d.mu.Unlock()

        // 5. Send
        SendDatumRequest(d.server.Conn, d.peerAddr, hash)
    }
}
```

2.3.3 Contrôle de congestion adaptatif

La taille de la fenêtre s'ajuste dynamiquement selon les performances du réseau :

```go
func (d *Downloader) responseLoop() {
    for hash := range d.responseCh {
        d.mu.Lock()
        sentAt, ok := d.pending[hash]
        if ok {
            // Succes !
            d.successCount++
            d.failureCount = 0

            // Augmenter la fenetre si reponse rapide
            if time.Since(sentAt) < d.timeout/2
               && d.windowSize < d.maxWindowSize {
                d.windowSize++
            }
            delete(d.pending, hash)
```

```
            delete(d.retries, hash)
        }
        d.mu.Unlock()

        // Traiter les enfants (recursion)
        if datum, ok := d.server.Downloads.Get(hash); ok {
            d.processChildren(datum)
        }
    }
}
```

Ce mécanisme offre plusieurs avantages :

— **Adaptation automatique** : La fenêtre augmente si le réseau est performant
— **Réduction lors d'échecs** : La fenêtre diminue en cas de timeouts
— **Limites configurables** : Fenêtre entre 1 et 100 paquets

2.3.4 Gestion des timeouts et retransmissions

Un worker de monitoring vérifie périodiquement les requêtes en attente et retransmet celles qui ont expiré :

```
// Verifier les timeouts
d.mu.Lock()
for hash, sentAt := range d.pending {
    if time.Since(sentAt) > d.timeout {
        // Timeout detecte
        retryCount := d.retries[hash]
        if retryCount >= MaxRetries {
            // Abandonner
            delete(d.pending, hash)
            delete(d.retries, hash)
            d.failureCount++
            d.windowSize = max(d.minWindowSize, d.windowSize-1)
        } else {
            // Retransmission
            d.retries[hash]++
            d.pending[hash] = time.Now()
            SendDatumRequest(d.server.Conn, d.peerAddr, hash)
        }
    }
}
d.mu.Unlock()
```

2.3.5 Téléchargement récursif

Le système télécharge récursivement tous les nœuds de l'arbre de Merkle :

1. Demande du hash racine au pair
2. Réception du datum racine
3. Analyse du type de nœud et extraction des hashes enfants
4. Ajout des hashes enfants à la file de téléchargement
5. Répétition jusqu'à avoir téléchargé tous les nœuds

Cette approche permet de télécharger des fichiers individuels ou des arborescences complètes selon les besoins de l'utilisateur.

Chapitre 3

Fonctionnalités avancées

3.1 Traversal NAT

L'une des fonctionnalités avancées implémentées est le NAT traversal, permettant à deux clients derrière des NAT différents de communiquer directement via un pair relais.

3.1.1 Principe du NAT traversal

Le mécanisme repose sur trois acteurs :
— **Source** : Le client qui souhaite établir une connexion
— **Cible** : Le client à atteindre derrière un NAT
— **Relais** : Un pair avec l'extension `NatTraversalRelay` qui coordonne la connexion

3.1.2 Protocole de traversal

Le protocole se déroule en plusieurs étapes :

1. Requête au relais La source envoie un message `NatTraversalRequest` au relais, contenant l'adresse de la cible :

```go
func (s *Server) onNatRequest(pkt *protocol.Packet, srcAddr *net.UDPAddr
    ) {
    targetStruct, err := protocol.DecodeSocketAddress(pkt.Body)
    if err != nil {
        return
    }

    // Verifier la signature de la source
    p, ok := s.PeerManager.GetByAddr(srcAddr)
    if !ok || !crypto.VerifySignature(p.PublicKey,
                    pkt.DataToSign(), pkt.Signature) {
        return
    }

    // Verifier si la cible est connue
    targetAddr := targetStruct.ToUDPAddr()
    targetPeer, targetKnown := s.PeerManager.GetByAddr(targetAddr)

    if !targetKnown {
        SendError(s.Conn, srcAddr, "Cible non connectee", pkt.Header.ID)
        return
    }

```

```go
    // OK a la source
    SendOk(s.Conn, srcAddr, pkt.Header.ID)

    // Notifier la cible
    SendNatTraversalRequest2(s.Conn, targetAddr, srcAddr, s.PrivKey)
}
```

2. Notification à la cible Le relais envoie un `NatTraversalRequest2` à la cible, lui indiquant l'adresse de la source.

3. Hole punching La cible reçoit la notification et commence à envoyer des pings vers la source avec un backoff exponentiel :

```go
func (s *Server) onNatRequest2(pkt *protocol.Packet, relayAddr *net.
    UDPAddr) {
    srcStruct, err := protocol.DecodeSocketAddress(pkt.Body)
    srcAddr := srcStruct.ToUDPAddr()

    // Creer un canal pour detecter les receptions
    responseChan := make(chan *net.UDPAddr, 10)
    s.PingResponseChan = responseChan

    // Backoff exponentiel : 3 tentatives
    count := 3
    totalTimeout := CalExpo2Time(count)

    for i := range count {
        SendPing(s.Conn, srcAddr)

        // Attendre avec backoff (1s, 2s, 4s)
        if i < count-1 {
            waitTime := time.Duration(1<<uint(i)) * time.Second
            select {
            case <-stopSending:
                return // Succes
            case <-time.After(waitTime):
                // Continuer
            }
        }
    }
}
```

4. Établissement de la connexion Pendant ce temps, la source envoie également des pings vers la cible. Grâce au hole punching, les NAT créent des mappings temporaires permettant aux paquets de passer dans les deux sens.

3.1.3 Détection du succès

Chaque paquet reçu déclenche une notification sur le canal `PingResponseChan` :

```go
// Dans handlePacket
s.PingResponseMu.Lock()
if s.PingResponseChan != nil {
    select {
    case s.PingResponseChan <- addr:
    default:
```

```
    }
}
s.PingResponseMu.Unlock()
```

Dès qu'un paquet est reçu de l'adresse cible, le processus s'arrête avec succès.

3.1.4 Gestion des relais

Les pairs qui souhaitent servir de relais s'annoncent avec l'extension `NatTraversalRelay` dans leur message Hello :

```
isRelay := (extensions & protocol.ExtNatTraversalRelay) != 0
s.PeerManager.AddOrUpdate(name, addr, pubKey, isRelay)
```

Le `PeerManager` maintient cette information et peut fournir la liste des relais disponibles.

3.2 Interface utilisateur

L'application propose une interface en ligne de commande interactive permettant :

- La sélection du répertoire à partager
- La liste des pairs connectés avec leurs adresses IPv4/IPv6
- Le téléchargement interactif de fichiers depuis un pair
- L'affichage de l'arborescence des fichiers distants
- Le téléchargement sélectif de fichiers ou de répertoires entiers
- L'affichage des statistiques de transfert (vitesse, progression)

3.3 Gestion multi-protocoles IPv4/IPv6

Notre implémentation gère nativement les deux protocoles :

- **Dual-Stack** : Écoute simultanée sur IPv4 et IPv6
- **Enregistrement multiple** : Chaque client enregistre ses deux adresses
- **Ping multi-adresses** : Les keep-alive maintiennent tous les chemins réseau
- **Fallback automatique** : Basculement transparent si un protocole échoue

Le `PeerManager` stocke toutes les adresses de chaque pair :

```go
type PeerInfo struct {
    Name      string
    Addrs     []*net.UDPAddr // Support IPv4 et IPv6
    PublicKey *ecdsa.PublicKey
    LastSeen  time.Time
    IsRelay   bool
}
```